

Maestro / MPTK 統合 (仮想デバイス + アダプター)

このページでは、オプションの **Maestro / MidiPlayerTK (MPTK)** 統合レイヤーについて説明します。

この統合により、主に 2 つのワークフローが提供されます:

1. 仮想 MIDI 出力デバイス (シンク) としての MPTK

`MidiManager` のイベントを MPTK シンセにルーティングします (リアルタイム演奏)。

2. 仮想 MIDI 入力デバイス (ソース) としての MPTK

MPTK のコールバック (`MidiFilePlayer` や `MidiStreamPlayer` からのもの) を受け取り、あたかもデバイスから届いたかのように `MidiManager` に MIDI イベントを 注入 します。

用語解説: このプラグインにおける「仮想デバイス (virtual device)」とは、プラットフォーム標準の MIDI バックエンドを介さず、`MidiManager` のデバイスリストやイベントパイプラインに参加するソフトウェア上のエンドポイントを指します。

有効化 / 無効化 (コンパイル時)

この統合機能は、以下のスクリプト定義シンボルによって制御されます:

- `FEATURE_USE_MPTK`
- `MPTK_PRO` (Maestro Pro 機能: `OnMidiEvent` Pipeline、Writer、InnerLoop、ListPlayer、Spatializer 等)

この定義が **存在しない** 場合 (または MPTK がインストールされていない場合)、統合スクリプトは空のスタブとしてコンパイルされるため、プロジェクト全体のビルドは維持されます。Pro 専用 API はさらに `MPTK_PRO` でガードされます。

実装の配置は `Assets/MIDI/Scripts/Integrations/MPTK/` です (Chunity と同様に `Integrations/` 配下)。

有効にする方法

Unity で以下の設定を行います:

- **Project Settings** → **Player** → **Other Settings** → **Script Compilation** → **Scripting Define Symbols**
- `FEATURE_USE_MPTK` を追加します。
- Maestro Pro を使う場合は `MPTK_PRO` も追加します。

対応 Maestro バージョン

統合テストの基準は **Maestro / MidiPlayerTK 2.21.x** (Free / Pro) です。Free 経路は 2.15–2.20 でも動くことがありますが、Pro ヘルパー (ボイス一時ミュート、Orientation、コード進行、再生中 SoundFont エフェクトなど 2.17 以降 API) は 2.21 系 Pro を想定します。

ランタイムヘルパー（コンポーネント / MCP）

ヘルパー	役割	Define / MCP
MptkSynthEffectsController	SoundFont filter / reverb / chorus のランタイム調整	MPTK_PRO · midi-mptk-effects-configure
MptkVoiceLifecycle / MptkMidiDevice.PauseVoices	スムーズなボイス一時ミュート	MPTK_PRO · midi-mptk-voice-lifecycle
MptkDistanceAudioSettings	距離減衰 + Pro Orientation	Free 距離 · Pro Orientation · midi-mptk-distance-audio
MptkChordProgressionPlayer	Maestro 進行プリセット再生	MPTK_PRO · midi-mptk-chord-progression
グローバル設定（MptkUtility / MptkBootstrap）	MPTK_RunInBackground / MPTK_AudioListener（EnsureReady 時に任意適用可）	FEATURE_USE_MPTK · midi-mptk-global-settings
実験的 Velocity 曲線	MPTK_VelocityAttenuation（MptkUtility、注意して使用）	FEATURE_USE_MPTK · midi-mptk-velocity-attenuation
Visual Scripting / Timeline	Effects / Voice / Distance / Chord / Global ユニット、ストリーム制御マーカー	FEATURE_USE_VISUALSCRIPTING / FEATURE_USE_TIMELINE

診断: midi://mptk/setup-report。ドラムプリセット: MptkUtility.BuildDrumPresetsText()、midi://mptk/soundfont、または midi-mptk-bank-program の action=list-drums。

エフェクト / Orientation は CPU 負荷や体感音量に影響し得ます。不要ならオフのままにし、必要なら Global Volume を調整してください。

含まれているもの

仮想デバイスの登録と入力注入

`MidiManager` には、ソフトウェアエンドポイント用のヘルパー API が用意されています:

- 仮想デバイス ID の登録/解除 (入力用、出力用、またはその両方として)。
- MIDI イベントを内部パイプラインに直接注入 (「このデバイスが Note On を送信したことにする」など)。

デバイスをシミュレートしたり、他のシステムと橋渡ししたり、ハンドラのユニットテストを行いたい場合に使用します。

MPTK 仮想 MIDI デバイス (シンク)

`MptkMidiDevice` は MIDI イベントハンドラインターフェースを実装しており、イベントを内部の MPTK `MidiStreamPlayer` に転送します。

以下のような場合に使用します:

- `MidiManager` → MPTK シンセでの再生。
- 実際のデバイスと並んで表示される「ソフトウェア出力デバイス ID」として利用。

MPTK → MidiManager アダプター (ソース)

`MptkMidiManagerInputAdapter` は MPTK のコールバックイベント (MPTK プレイヤーから公開されるもの) を購読し、対応する MIDI 1.0 形式のイベントを `MidiManager` に注入できます。

以下のような場合に使用します:

- MPTK の再生やリアルタイム生成内容によって、既存の `MidiManager` ハンドラを動作させる。
- MPTK を仮想的な 入力 デバイスのように振る舞わせる。

便利なユーティリティ

`MptkUtility` は一般的なセットアップをラップしています:

- MPTK グローバル設定の存在確認。
- MPTK プレイヤーの作成。
- 仮想デバイスの作成と登録。
- アダプターの作成。
- MIDI イベントを仮想入力として注入するためのヘルパーメソッド。
- セットアップ検証 (`ValidateSetup`)。

合成前 rewrite (`MptkMidiEventPipeline` 、 **Maestro Pro**)

`OnMidiEvent` による keep / drop / inject。詳細は後述の Bootstrap / 診断節内「`MptkMidiEventPipeline`」を参照。

Writer / 外部 URI (`MptkWriterBridge` 、 `MptkExternalPlayback` 、 **Maestro Pro**)

`MPTKWriter` / `MidiExternalPlayer` の薄いラップ。詳細は後述の「`MptkWriterBridge` / `MptkExternalPlayback`」を参照。

内側ループ (`MptkInnerLoopController` 、 **Maestro Pro**)

`MPTK_InnerLoop` の薄いラップ。詳細は後述の Clock 同期節内「ループ領域」を参照。

Bootstrap / SmfPlayer / 診断

セットアップの簡略化、SMF 再生との接続、Panic / Bank Select、デバッグ支援を提供します。

MptkBootstrap — ワンクリック起動

MptkBootstrap は以下を自動で行います:

- MidiPlayerGlobal の確保
- 仮想デバイス ID の登録 (入出力)
- MptkMidiDevice の生成と MidiManager ハンドラ登録

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyMptkSetup : MonoBehaviour
{
    void Start()
    {
        var bootstrap = gameObject.AddComponent<MptkBootstrap>();
        bootstrap.EnsureReady();
    }
}
#endif
```

推奨デフォルト ID: MptkUtility.DefaultOutputDeviceId (mptk:internal)

MptkSmfPlayerOutput — SmfPlayer 連携

SmfPlayer の outputDeviceId を MPTK 仮想デバイスへ自動設定します。

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi;
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

[RequireComponent(typeof(SmfPlayer))]
public sealed class MySmfWithMptk : MonoBehaviour
{
    void Awake()
    {
        var output = gameObject.AddComponent<MptkSmfPlayerOutput>();
        output.Configure(); // SmfPlayer.outputDeviceId = "mptk:internal"
    }
}
#endif
```

同一 GameObject に MptkBootstrap を置くと、MptkSmfPlayerOutput がそれを再利用します。

MptkDspUmpSequenceOutput — Scriptable Audio UMP 連携

FEATURE_USE_MPTK と FEATURE_SCRIPTABLE_AUDIO の両方が有効なとき、MidiDspUmpMidi2OutBridge 経由の DSP スケジュール済み UMP を MPTK 仮想出力へルーティングできます。

ソースは Assets/MIDI/Scripts/Integrations/MPTK/ScriptableAudio/ (アセンブリ jp.kshoji.midi.mptk.scriptableaudio) にあり、Chunity / Timeline と同様のオプション・サブフォルダ構成です。

1. UMP Bootstrap / Scheduler と同じ GameObject (または配線先) に MptkDspUmpSequenceOutput を追加
2. 必要なら MptkBootstrap を同居させる (欠落時は自動作成オプションあり)
3. Play モードで Configure() (または autoConfigureOnAwake) が UMP Out の出力先を mptk:internal 等に設定

内蔵参照シンセと二重に鳴らしたくない場合は、コンポーネントの muteBuiltInSynthWhenActive を有効にします。詳細は [UMP シークス再生 \(DSP 同期\)](#) を参照してください。

Panic / All Notes Off / Reset

MptkMidiDevice.PanicAll() は以下を実行します:

- 追跡中の Note On を MPTK_StopEvent で停止
- 全チャンネルへ CC 120 (All Sound Off) / CC 123 (All Notes Off)

CC 120 / 121 / 123 を受信した場合も、チャンネル単位で同等処理を行います。MIDI Reset (OnMidiReset) では PanicAll() を呼び出します。

MptkBootstrap.Panic() から同操作を実行できます。出力先 deviceId を mptk:internal 等に設定してください。

Bank Select + Program Change

MptkMidiDevice / MptkMidi2Device は CC 0 (Bank MSB) / CC 32 (Bank LSB) をチャンネルごとに追跡し、Program Change 時に MPTKController.BankSelectMsb + MPTKCommand.PatchChange を MPTK へ送信します。General MIDI 以外の SoundFont でも音色切替が安定します。

セットアップ検証 (MptkUtility.ValidateSetup)

Play モード中に MPTK 連携の状態を確認できます。

```
var report = MptkUtility.ValidateSetup(bootstrap.Sink, bootstrap.DeviceId);
Debug.Log(report.ToSummary());
```

チェック項目	重大度
MidiPlayerGlobal 存在	Error
MidiManager 利用可能	Error
仮想デバイス登録	Error
MidiStreamPlayer / AudioSource	Error / Warning
SoundFont ロード	Error
適用済み SF 名 (expectedSoundFontName 指定時)	Warning

MptkBootstrap は autoValidateOnStart 有効時、起動時に自動検証します。

デバッグ: MptkEventTap

ブリッジ上の MIDI イベントを Console に記録するオプションコンポーネントです。

プロパティ	説明
direction	ToMptk (シンク方向) / FromMptk (アダプタ注入方向)
filterDeviceId	特定 deviceId のみログ
logNoteEvents / logControlChange	イベント種別フィルタ

シーンに 1 つ追加し、isLoggingEnabled = true にすると MptkMidiDevice / MptkMidiManagerInputAdapter からのログが有効になります。

MptkMidiEventPipeline — 合成前 rewrite (Maestro Pro)

MidiFilePlayer / MidiExternalPlayer の OnMidiEvent をフックし、SMF を編集せずに keep / drop / inject できます。

項目	内容
定義	FEATURE_USE_MPTK + ** MPTK_PRO ** (OnMidiEvent / PlayDirect は Pro)
コンポーネント	MptkMidiEventPipeline
Mapping SO	Create > MIDI > MPTK > Event Mapping (MptkMidiEventMapping)
サンプル	Assets/MIDI/Samples/MPTK/Scripts/MptkMidiEventPipelineSample.cs

内蔵トグル例: アルペジオ inject、PatchChange 破棄、SetTempo ランダム化。コード側は Filter (非メインスレッド — Unity API 禁止)。Mapping の UnityEvent はメインスレッドへキューイングされます。

```

#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using MidiPlayerTK;
using UnityEngine;

public sealed class MyPipelineSetup : MonoBehaviour
{
    public MidiFilePlayer filePlayer;

    void Start()
    {
        var pipeline = gameObject.AddComponent<MptkMidiEventPipeline>();
        pipeline.Source = filePlayer;
        pipeline.EnableArpeggio = true;
        pipeline.Filter = e =>
            e.Command == MPTKCommand.NoteOn && e.Channel == 9
                ? MptkMidiEventPipeline.Result.Drop
                : MptkMidiEventPipeline.Result.Keep;
    }
}
#endif

```

InputAdapter と併用するときは、rewrite 後のイベント（`OnEventNotesMidi`）だけを仮想入力へ流し、二重発音を避けてください。

MptkWriterBridge / MptkExternalPlayback — Writer / 外部 URI（Maestro Pro）

項目	内容
定義	<code>FEATURE_USE_MPTK + ** MPTK_PRO **</code> （ <code>MPTKWriter</code> / <code>MidiExternalPlayer</code> ）
Writer	<code>MptkWriterBridge</code> — <code>MidiSequenceAsset</code> / SMF bytes / <code>MidiDB</code> / <code>ImportFromPlayer</code> → Write / in-memory Play
External	<code>MptkExternalPlayback</code> — <code>file://</code> / <code>http(s)://</code> 再生、任意で <code>InputAdapter</code> 接続（ <code>ConnectInputAdapterOnPlay</code> ）
サンプル	<code>Assets/MIDI/Samples/MPTK/Scripts/MptkWriterExternalSample.cs</code> （生成 / Join / temp <code>.mid</code> / URI）

```

#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyWriterExternalSetup : MonoBehaviour
{
    public MidiSequenceAsset sequence;

    void Start()
    {
        var writer = gameObject.AddComponent<MptkWriterBridge>();
        var external = gameObject.AddComponent<MptkExternalPlayback>();
        writer.LoadFromSequenceAsset(sequence);
        var path = writer.WriteToTempFile();
        external.ConnectInputAdapterOnPlay = true;
        external.PlayFile(path);
    }
}
#endif

```

プレイリスト / SoundFont / 遅延ディスパッチ / チャンネル劇場

コンポーネント	根拠 Demo	Pro ガード	サンプル
MptkListPlayerBridge	TestMidiListPlayer	MidiListPlayer → MPTK_PRO	MptkListPlayerSample.cs
MptkSoundFontLoader	TestLoadSF	ランタイム Load → MPTK_PRO	MptkPhaseDUilitiesSample.cs
MptkDelayedNoteDispatcher	CatchMusic	なし (OnEventNotesMidi)	同上
MptkFilePlayerChannels	MidiChannel*	PlayDirect / StopDirect → MPTK_PRO	同上

List: SetPlaylist / AddMidi / PlayAtIndex / OverlayTimeMs 。曲開始時に ConnectInputAdapterOnSongStart で InputAdapter へ接続可能。

SoundFont: URL / file:// / StreamingAssets / 内蔵名を Load 。MptkUtility.ValidateSetup(..., expectedSoundFontName: "...") で適用名を検証。

Delayed: mute した FilePlayer の OnEventNotesMidi を ms または tick オフセットでキューし、Stream または MidiManager へ送出。視覚デモは同梱せず契約のみ。

Channels: SetChannelEnabled / SetSoloChannel / SetDrumsOnly / SetSustain / PlayDirect / StopDirect / Panic 。

Spatializer / Visual Scripting / Timeline

コンポーネント	根拠 Demo	Pro / Define	サンプル
MptkSpatializerHost	SimplestMidiSpatializer	MidiSpatializer → MPTK_PRO	MptkSpatializerSample.cs
MptkSpatializerLayout	—	deviceId / 位置 SO	—
MptkDistanceAudioSettings	—	DistanceAttenuation + Orientation (Pro)	MCP midi-mptk-distance-audio
VS ノード (Pipeline 等)	P2	FEATURE_USE_VISUALSCRIPTING	Window メニューで登録
MptkFilePlayerMarker + Receiver	Seek / InnerLoop	FEATURE_USE_TIMELINE	MptkTimelineMarkerExample.cs

Maestro Pro の **MidiSpatializer** prefab を Host に割り当ててください。聴覚上の 3D には Unity 空間化プラグインも必要です。各シ
ンセのノートは `deviceIdPrefix:index` (または Layout のサフィックス) で **MidiManager** へ注入されます。

レイアウト SO: Create > MIDI > MPTK > Spatializer Layout (MptkSpatializerLayout) 。

距離減衰と Spatializer の違い

これらは 別機能 です。

機能	コンポーネント / API	役割
Spatializer (Track/Channel)	MptkSpatializerHost + Maestro MidiSpatializer	1 つの MIDI → 多数の synth (track または channel 単位) を 3D アンカーに配置
距離減衰	MptkDistanceAudioSettings / MPTK_DistanceAttenuation	単一 synth の音量をリスナー距離で変化 (min/max、距離超過で pause)
Orientation (Pro)	同上 / MPTK_Orientation	MPTK_AudioListener との角度によるパン・前後フィルタ

マルチ楽器レイアウトには Spatializer。Stream/File (または各 spatial synth) が距離・角度に反応するときは
MptkDistanceAudioSettings (またはホストの **Arrange** 時 **Distance / Orientation**)。MCP: midi-mptk-distance-audio 。

コード進行の生成と和音認識の違い

機能	API	役割
進行の生成 (MPTK Pro)	MptkChordProgressionPlayer / midi-mptk-chord-progression	Maestro の感情プリセット進行を MidiStreamPlayer で再生
和音認識 (キット)	ChordRecognition / midi-chord-state	押下中の入力ノートから和音名を推定

生成は発音、認識は入力解析で、置き換えではありません。

ドラムプリセット一覧: midi://mptk/soundfont および midi-mptk-bank-program の action=list-drums 。

MptkIntegrationSampleScene に Pipeline / Writer / InnerLoop の任意 GUI トグル (加えて Effects / Voice / Distance / Chord) を追加済みです。

サンプルシーン / 出力プリセット / エディタ試聴

専用サンプルシーン

項目	パス
シーン	Assets/MIDI/Samples/MPTK/Scenes/MptkIntegrationSampleScene.unity
スクリプト	Assets/MIDI/Samples/MPTK/Scripts/MptkIntegrationSampleScene.cs

FEATURE_USE_MPTK 有効時に以下を GUI から試せます:

- MptkBootstrap セットアップ / 検証 / Panic
- MidiOutputRoutingPreset 経由の Note 送信
- SmfPlayer + MptkSmfPlayerOutput による SMF 試聴
- Pipeline (arp) / Writer (demo ノート or SequenceAsset) / InnerLoop 区間 / Effects / Voice / Distance / Chord

Inspector で outputPreset に Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset を割り当ててください。

キット共通 outputDeviceId プリセット

MidiOutputRoutingPreset (Assets > Create > MIDI > Output Routing Preset) と MidiOutputRouting ヘルパーで、コンポーネント間で MPTK 出力先を共通化できます。

outputDeviceId / outputPreset フィールドを持つ送信コンポーネントで利用できます。

解決順序: outputDeviceId (明示) > outputPreset > 最初の出力デバイス

同梱プリセット:

- Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset — mptk:internal ヘルパーティング

```
using jp.kshoji.unity.midi.foundation;

// キット送信例
MidiOutputRouting.CreateBuilder(outputDeviceId, outputPreset, group)
    .Channel(channel)
    .NoteOn(60, 100);
```

エディタ SMF 試聴 (Window > MIDI > SMF Preview)

SMF Preview ウィンドウに **Preview Audio (MPTK)** ボタンを追加しました (FEATURE_USE_MPTK 必須)。

1. .mid を読み込む
2. **Preview Audio (MPTK)** をクリック → Play Mode に入り SmfPlayer が MPTK 仮想出力で再生
3. **Stop Preview** で停止

内部コンポーネント:

コンポーネント	役割
SmfPreviewPlaybackRequest	エディタ → Play Mode への再生リクエスト
SmfPreviewPlaybackHost	Play Mode で SmfPlayer を起動
MptkSmfPreviewPlaybackHook	MPTK Bootstrap を先行初期化

Clock 同期 / ループ領域 / MIDI 2.0 入力 / MPE / Visual Scripting

Clock 同期 — MptkClockSyncBridge

外部 MidiClockSync と MPTK 再生（SmfPlayer / MidiFilePlayer）を連携します。

モード	動作
Follow	推定 BPM を SmfPlayer.tempoBpm / MidiFilePlayer.MPTK_Tempo に反映
Step	外部 Clock の拍ごとに SmfPlayer を 1 拍進める
Free	外部 Clock を無視

syncTransportToClock を有効にすると、外部 Start / Stop に合わせて SMF / MPTK ファイル再生を開始・停止します。

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi;
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyClockBridge : MonoBehaviour
{
    public MidiClockSync clockSync;
    public SmfPlayer smfPlayer;

    void Awake()
    {
        gameObject.AddComponent<MptkBootstrap>().EnsureReady();
        gameObject.AddComponent<MptkSmfPlayerOutput>().Configure();
        gameObject.AddComponent<MptkClockSyncBridge>();
    }
}
#endif
```

MPTK コールバックから Clock イベントを MidiManager に注入する場合は、既存の MptkMidiManagerInputAdapter が Timing Clock / Start / Stop / Continue を転送します。

ループ領域 — MptkInnerLoopController (Maestro Pro)

MptkClockSyncBridge は 外部 Clock のテンポ／トランスポート同期です。一方 MptkInnerLoopController は **MPTK FilePlayer / ExternalPlayer** 再生中の内側ループ（MPTK_InnerLoop）をラップします。SmfPlayer や DSP Scheduler の loop とも別エンジンです。

項目	内容
定義	<code>FEATURE_USE_MPTK + ** MPTK_PRO ** (MPTK_InnerLoop)</code>
コンポーネント	<code>MptkInnerLoopController</code> — <code>Start / Resume / End / Max / Finished</code>
イベント	<code>OnLoopStart / OnLoopResume / OnLoopExit</code> (メインスレッド)。 <code>PhaseFilter</code> は MIDI スレッド (Unity API 禁止)
ヘルパ	<code>MeasureToTick / SetLoopByMeasure</code> (拍子→tick)
サンプル	<code>Assets/MIDI/Samples/MPTK/Scripts/MptkInnerLoopSample.cs</code>

MIDI ロード時に `MPTK_InnerLoop` はクリアされるため、デフォルトで `OnEventStartPlayMidi` 時にパラメータを再適用します (`ReapplyOnStartPlay`)。

Free フォールバック: Pro が無い場合は `MPTK_MidiLoaded.MPTK_TickStart / MPTK_TickEnd + MPTK_MidiAutoRestart` で粗い区間リスタートが可能です (`MidiLoop` デモ相当)。精度・位相コールバックが必要なら `InnerLoop` を使ってください。

```
#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using MidiPlayerTK;
using UnityEngine;

public sealed class MyInnerLoopSetup : MonoBehaviour
{
    public MidiFilePlayer filePlayer;

    void Start()
    {
        var loop = gameObject.AddComponent<MptkInnerLoopController>();
        loop.Source = filePlayer;
        // Start → Resume ... → End (Max 回) ; Max=0 は無限
        loop.SetLoop(start: 0, resume: 480 * 4, end: 480 * 16, max: 3);
        loop.OnLoopExit.AddListener(() => Debug.Log("chorus loop done"));
        filePlayer.MPTK_Play();
    }
}
#endif
```

MIDI 2.0 入力アダプタ — `MptkMidi2ManagerInputAdapter`

MPTK プレイヤーのコールバックを `Midi2Manager` に **MIDI 2.0 仮想入力**として注入します (デフォルト `deviceId: mptk2:internal`)。

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi.mptk;

MptkUtility.CreateMidi2ManagerInputAdapter(
    filePlayer,
    streamPlayer,
    deviceId: MptkUtility.DefaultMidi2InputDeviceId);
#endif
```

MIDI 1.0 7-bit 値は UMP 向けに 16-bit / 32-bit ヘスケールして注入されます。

MPE 出力 — MptkMpeOutput

MpeManager 経由で MPTK 仮想デバイスへ MPE ノートを送信します。内部で SetupMpeZone を呼び、メンバーチャンネル割り当ては MpeManager が担当します。

```
#if FEATURE_USE_MPTK
var mpe = gameObject.AddComponent<MptkMpeOutput>();
mpe.ConfigureZone();
mpe.SendNoteOn(60, 100);
#endif
```

Visual Scripting ノード (FEATURE_USE_MPTK + FEATURE_USE_VISUALSCRIPTING)

アセンブリ jp.kshoji.midi.mptk.visualscripting に MPTK 専用ノードを追加しました。

ノード	カテゴリ	説明
MPTK Ensure Ready	MIDI/MPTK	<code>MptkBootstrap.EnsureReady()</code>
MPTK Panic	MIDI/MPTK	All Notes Off
MPTK Send Routed Note On	MIDI/MPTK	<code>MidiOutputRoutingPreset</code> 経由の Note On
MPTK Configure MPE Zone	MIDI/MPTK	<code>MptkMpeOutput.ConfigureZone()</code>
MPTK Pipeline Configure	MIDI/MPTK	Pipeline の arp / drop patch / tempo
MPTK Writer Play	MIDI/MPTK	<code>MptkWriterBridge.Play()</code>
MPTK External Play	MIDI/MPTK	<code>MptkExternalPlayback.Play(uri)</code>
MPTK InnerLoop Apply	MIDI/MPTK	<code>MptkInnerLoopController.SetLoop</code>
MPTK List Play	MIDI/MPTK	<code>MptkListPlayerBridge.Play</code> / <code>PlayAtIndex</code>
MPTK SoundFont Load	MIDI/MPTK	<code>MptkSoundFontLoader.Load</code>
MPTK Spatializer Play	MIDI/MPTK	<code>MptkSpatializerHost.Play</code>
MPTK Effects Apply	MIDI/MPTK	SoundFont filter / reverb / chorus
MPTK Voice Lifecycle	MIDI/MPTK	ボイス一時ミュート / 復帰
MPTK Distance Audio Apply	MIDI/MPTK	距離 / Orientation
MPTK Chord Progression	MIDI/MPTK	進行プリセットの再生 / 停止
MPTK Global Settings	MIDI/MPTK	<code>SetRunInBackground</code>

初回セットアップ:

1. `FEATURE_USE_MPTK` と `FEATURE_USE_VISUALSCRIPTING` を有効化
2. **Window > MIDI > Visual Scripting > Register MPTK Nodes** を実行
3. Script Graph で **MIDI/MPTK** カテゴリのノードを使用

ドキュメント例

例	パス
Clock 同期	<code>Assets/MIDI/Samples/DocumentationExamples/MptkClockSyncExample.cs</code>
MIDI 2.0 入力	<code>Assets/MIDI/Samples/DocumentationExamples/MptkMidi2InputAdapterExample.cs</code>
MPE 出力	<code>Assets/MIDI/Samples/DocumentationExamples/MptkMpeOutputExample.cs</code>
Event Pipeline	<code>Assets/MIDI/Samples/DocumentationExamples/MptkMidiEventPipelineExample.cs</code>
Writer / External	<code>Assets/MIDI/Samples/DocumentationExamples/MptkWriterExternalExample.cs</code>
InnerLoop	<code>Assets/MIDI/Samples/DocumentationExamples/MptkInnerLoopExample.cs</code>
List / SF / Delayed / Channels	<code>MptkListPlayerExample.cs</code> / <code>MptkSoundFontLoaderExample.cs</code> / <code>MptkDelayedNoteDispatcherExample.cs</code> / <code>MptkFilePlayerChannelsExample.cs</code>
Spatializer	<code>Assets/MIDI/Samples/DocumentationExamples/MptkSpatializerExample.cs</code>
Timeline Marker	<code>Assets/MIDI/Samples/DocumentationExamples/MptkTimelineMarkerExample.cs</code>

以下のサンプルが `Assets/MIDI/Samples/DocumentationExamples/` に用意されています:

- **MIDI 1.0 → MPTK (仮想出力シンク)**

`Assets/MIDI/Samples/DocumentationExamples/MptkVirtualOutputSinkExample.cs`

- **MPTK → MIDI 1.0 (仮想入力として `MidiManager` に注入)**

`Assets/MIDI/Samples/DocumentationExamples/MptkToMidiManagerInputExample.cs`

- **Bootstrap + SmfPlayer + 検証**

`Assets/MIDI/Samples/DocumentationExamples/MptkBootstrapExample.cs`

- **Clock 同期**

`Assets/MIDI/Samples/DocumentationExamples/MptkClockSyncExample.cs`

- **合成前 Event Pipeline**

`Assets/MIDI/Samples/DocumentationExamples/MptkMidiEventPipelineExample.cs`

- **Writer / External 往復**

`Assets/MIDI/Samples/DocumentationExamples/MptkWriterExternalExample.cs`

- **InnerLoop (ループ領域)**

`Assets/MIDI/Samples/DocumentationExamples/MptkInnerLoopExample.cs`

- **List / SoundFont / Delayed / Channels**

`Assets/MIDI/Samples/DocumentationExamples/MptkListPlayerExample.cs` 他

- **Spatializer**

`Assets/MIDI/Samples/DocumentationExamples/MptkSpatializerExample.cs`

- **Timeline Marker Seek / InnerLoop**

`Assets/MIDI/Samples/DocumentationExamples/MptkTimelineMarkerExample.cs`

- **MIDI 2.0 入力アダプタ**

`Assets/MIDI/Samples/DocumentationExamples/MptkMidi2InputAdapterExample.cs`

- **MPE 出力**

`Assets/MIDI/Samples/DocumentationExamples/MptkMpeOutputExample.cs`

サンプルシーン (統合済み)

同梱のサンプルシーンにも、オプションで MPTK 統合が含まれています:

- MIDI 1.0 サンプルシーンスクリプト:

`Assets/MIDI/Samples/Scripts/MidiSampleScene.cs`

(MPTK バックエンドの仮想出力デバイスを使用するトグルを追加します)

- MIDI 2.0 サンプルシーンスクリプト:

`Assets/MIDI/Samples/Scripts/Midi2SampleScene.cs`

(MIDI 2.0 の送信内容を MPTK MIDI 1.0 仮想シンクに ミラーリングするトグルを追加します)

ID、グループ、ルーティングの選択

推奨される慣例:

- 仮想デバイスであることが明確なプレフィックスを使用してください（例: `mptk:internal`, `virtual:sequencer`, `test:device`）。
- 意図的に複数のグループをモデル化する場合を除き、`group = 0` を使用してください。

仮想デバイスはデバイスセット内に表示されるため、ハードウェアデバイスと同じようにユーザーが UI から選択できる仕組みを構築できます。

トラブルシューティング

「エディタではコンパイルできるが、CI や他のマシンで失敗する」

- `FEATURE_USE_MPTK` は、実際に MPTK アセットが存在する場合にのみ有効にしてください。

「イベントが受信されない」

- 注入されたイベントが入力デバイスからのものとして認識されるよう、仮想デバイスが **入力 (input)** として登録されているか確認してください。
- ハンドラが `MidiManager` に登録されているか確認してください。
- `deviceId` と `group` が期待通りであることを確認してください。

「MPTK シンクから音が鳴らない」

- 仮想デバイスが **出力 (output)** として登録されているか確認してください。
- シンクの `deviceId` が、送信先のデバイス ID と一致しているか確認してください。
- プロジェクト内の MPTK グローバル設定やリソース (SoundFont/構成) が有効であることを確認してください。
- `MptkUtility.ValidateSetup()` または `MptkBootstrap.Validate()` で SoundFont / AudioSource / 仮想デバイス登録を確認してください。

「SmfPlayer から MPTK に音が出ない」

- 同じ GameObject (または子) に `MptkSmfPlayerOutput` を追加し `Configure()` を呼び出してください。
- `SmfPlayer.outputDeviceId` が `mptk:internal` (または Bootstrap で設定した ID) になっているか確認してください。
- Play モード開始前に `MptkBootstrap.EnsureReady()` が完了している必要があります。

「Scriptable Audio UMP から MPTK に音が出ない」

- `FEATURE_SCRIPTABLE_AUDIO` と `FEATURE_USE_MPTK` の両方が有効か確認してください。
- `MptkDspUmpSequenceOutput` を追加し、`MidiDspUmpMidi2OutBridge` が有効であること (Bootstrap の UMP Out または手動配線) を確認してください。
- `MidiManager.InitializeMidi2()` が呼ばれているか確認してください。

「Panic しても音が残る」

- `MptkBootstrap.Panic()` または `MptkMidiDevice.PanicAll()` を直接呼び出してください。
- ハードウェアと MPTK の両方へ送っている場合、それぞれの出力先で Panic が必要です。

関連ドキュメント

- [MIDI 1.0 \(MidiManager\)](#)
- [Unity エコシステム統合 — UMP シーケンス](#)
- [サンプル](#)